

AI Smart Complaint Resolver

English-Only Community Complaint Classification, Triage & Civic Routing System

Author / Team: Civic AI Engineering	Version: 1.0.0 (Production-Ready)
Domain: Natural Language Processing (NLP)	Date: September 2026

1. Executive Summary

Modern municipalities receive hundreds of daily civic complaints spanning broken water mains, live electrical hazards, massive potholes, and illegal waste dumping. Manual grievance triage leads to severe operational bottlenecks, misrouted work orders, and sluggish emergency response times.

The **AI Smart Complaint Resolver** provides an end-to-end, automated machine learning pipeline tailored for English community complaints. It ingests unstructured complaint text, predicts the civic **Category** across 8 domains, determines operational **Urgency** across 4 priority tiers, routes the ticket to the appropriate municipal department, and generates a structured operational dispatch summary.

2. Dataset Architecture & Inspection

The dataset is stored at dataset/complaints.csv with 160 balanced records, comprising three primary fields:

Field Name	Data Type	Specification / Scope
Message	Text (String)	Raw citizen grievance describing the incident or infrastructure failure.
Category	Categorical (8)	Flood, Waste, Road, Drainage, Water, Electricity, Infrastructure, Pollution (20 records each).
Urgency	Categorical (4)	Low, Medium, High, Critical (40 records each across categories).

3. Machine Learning Pipeline Architecture

The system employs a dual-classifier architecture implemented using scikit-learn. Rather than relying on heavy deep neural networks or external paid APIs, the pipeline leverages high-performance TF-IDF vectorization paired with calibrated linear classifiers for fast, reproducible inference.

A. Feature Extraction (TF-IDF Vectorizer)

- **N-Gram Range (1, 2):** Captures both single words (*pothole, transformer*) and essential multi-word domain collocations (*power outage, water burst, flash flood*).
- **Sublinear TF Scaling:** Replaces raw term frequency with $1 + \log(\text{tf})$, dampening the impact of repeated words.
- **Stop Words Removal:** Strips English stop words to emphasize domain-specific civic terminology.

B. Model Training & Serialization

- **Category Classifier:** LogisticRegression(class_weight='balanced', max_iter=1000) predicts civic domain across 8 classes.
- **Urgency Classifier:** LogisticRegression(class_weight='balanced') calibrated to prevent critical safety hazards from being misclassified as routine.
- **Model Artifacts:** Serialized using joblib into models/category_model.joblib and models/urgency_model.joblib.

4. Civic Department Routing & Dispatch Summary Engine

Once a complaint is classified, src/resolver.py executes deterministic routing against the municipal directory, mapping the predicted category and urgency to the responsible department and generating an operational dispatch summary.

Category	Routing Department	Code	Recommended Action
Flood	Disaster Management & Stormwater Div.	CIVIC-FLD-101	Deploy water-pumping teams & flood rescue.
Drainage	Public Works - Drainage & Sewerage Board	CIVIC-DRN-102	Deploy clearance crew & sewer jetting units.
Water	Municipal Water Supply & Treatment Authority	CIVIC-WTR-103	Inspect pipelines & send emergency tankers.
Waste	Solid Waste Management & Sanitation Dept.	CIVIC-WST-104	Dispatch compactor trucks & sanitation crew.
Road	Roads, Bridges & Transportation Authority	CIVIC-ROD-105	Send repair team for patching & barricading.
Electricity	State Electricity Board / Power Grid Div.	CIVIC-ELE-106	Notify substation linesmen & rapid response.
Infrastructure	Urban Infrastructure & Civil Engineering	CIVIC-INF-107	Assign structural engineers for safety review.
Pollution	Environmental Protection & Pollution Control	CIVIC-POL-108	Dispatch inspectors for air/water testing.

5. Experimental Results & End-to-End Verification

The model was evaluated using an 80/20 train/test split. An automated test suite (tests/test_resolver.py) verified end-to-end inference across unseen test complaints.

Test Complaint Excerpt	Expected	Predicted	Urgency	Status
Continuous rain caused flash water surge in colony...	Flood	Flood	Critical	PASS (100%)
Mountain of uncollected garbage rotting on corner...	Waste	Waste	High	PASS (100%)
Deep dangerous pothole on highway caused accidents...	Road	Road	High	PASS (100%)
Raw sewage overflowing directly from manhole...	Drainage	Drainage	High	PASS (100%)
Main drinking water pipeline fractured on 8th Street...	Water	Water	High	PASS (100%)
High voltage electrical wire snapped and sparking...	Electricity	Electricity	Critical	PASS (100%)

Concrete railing of pedestrian bridge cracked...	Infrastructure	Infrastructure	High	PASS (100%)
Industrial factory discharging toxic sulfur smoke...	Pollution	Pollution	High	PASS (100%)

6. Streamlit Web Application Interface

An interactive dashboard was built using Streamlit (app.py), running locally on http://localhost:8501. The application features:

- 1. **Text Area Input:** Allows citizens and municipal operators to paste complaints in English.
- 2. **Sidebar Presets:** 1-click test buttons preloaded with realistic civic issues.
- 3. **Category & Urgency Badges:** Color-coded visual badges with confidence percentage.
- 4. **Department Routing Directive:** Highlights recommended actions and civic dispatch codes.
- 5. **Short AI-Generated Summary:** Delivers a standardized executive briefing for municipal dispatchers.

7. Repository Structure

```
d:\ai_compliant\  
|-- dataset/  
|   +-- complaints.csv           # 160 balanced English community complaints  
|-- models/  
|   |-- category_model.joblib    # TF-IDF + LogisticRegression model (Category)  
|   +-- urgency_model.joblib     # TF-IDF + LogisticRegression model (Urgency)  
|-- src/  
|   |-- __init__.py  
|   |-- data_loader.py           # Data loader, cleaner, and validation  
|   |-- train.py                 # Model training and evaluation script  
|   +-- resolver.py             # Inference, department routing & summary generator  
|-- tests/  
|   +-- test_resolver.py         # Automated end-to-end test suite  
|-- app.py                      # Streamlit web application  
|-- requirements.txt             # Dependencies (pandas, scikit-learn, joblib, streamlit)  
+-- README.md                   # Comprehensive user & setup guide
```

8. Conclusion & Future Scope

The **AI Smart Complaint Resolver** successfully establishes an automated, beginner-friendly, and robust English grievance analysis system. By categorizing civic domain, estimating urgency, and synthesizing operational summaries, it provides immediate civic value without external dependencies. Future enhancements may include geo-tagging integration, image-based damage classification, and automated citizen SMS status notifications.